

50.057 Predicate Logic

ISTD, SUTD

Learning Outcomes

1. List the difference between predicate logic and propositional logic
2. List the syntax elements of predicate logic
3. Apply deduction proof in simple predicate logic
4. List the difference between simple predicate logic and first order logic
5. Apply linear induction proof in first order logic

Predicate Logic

Predicate Logic is an alternative to the propositional logic that supports variables and their quantification.

Let's begin with a motivating example.

Motivating Example

Suppose we want to use logic to express the system that if person A knows person B, then B knows A. To do that in propositional logic, we have to encode this requirement by enumerating

ada_knows_beatrice
beatrice_knows_ada
ada_knows_cara
cara_knows_ada
...

It is simply an inadequate representation.

A Better Representation

It would be nice if we could say

$$\forall x \in \textit{Obj}. (\forall y \in \textit{Obj}. (\textit{knows}(x, y) \implies \textit{knows}(y, x)))$$

Where *Obj* defines a **finite** set of objects.

$$\textit{Obj} = \{ada, beatrice, cara, daphne\}$$

Syntax of Predicate Logic

In predicate logic, instead of using “hard-coded” atoms as simple predicates, we define a set of objects.

Variables such as x , y and z are quantified from their domains. In this case, their domain must be *Obj*.

- ▶ “ $\forall$ ” reads as “for all”.
- ▶ A universal quantifier followed by a variable declares a universally quantified sentence.
- ▶ For brevity, we drop the domain of a variable when no confusion arises.

Universally Quantified Sentences

For example

$$\forall x.(\forall y.(knows(x, y) \implies knows(y, x)))$$

introduces two universally quantified variables x and y , where x 's scope is

$$\forall y.(knows(x, y) \implies knows(y, x))$$

and y 's scope is

$$(knows(x, y) \implies knows(y, x))$$

Existential Quantifier

Besides universal quantifier, the existential quantifier is supported in predicate logic.

$$\exists z.(\textit{knows}(\textit{ada}, z) \wedge \textit{knows}(\textit{cara}, z))$$

which says there exists a person whom both Ada and Cara know.

Predicates

On the other hand, *knows*(_, _) is a predicate.

- ▶ A predicate cannot be an object and vice versa.
- ▶ A predicate defines some association between objects.
- ▶ Note that predicates can be unary, and n-ary too.

Implicit Restrictions

There are three non-syntactical implicit restrictions (for the time being).

1. Objects must be finite, violating which leads us to first-order logic
2. Predicate's argument must be a **simple object** or variable, violating which leads us to first-order logic.
3. A variable's domain must be objects, violating which leads us to higher-order logic.

Precedence

The precedence level of predicate logic is the same as the one of propositional logic, except that the universal and existential quantifiers have the **strongest** precedence level.

For example, $\exists x.good(x) \wedge \exists y.bad(y)$ is parsed as $(\exists x.good(x)) \wedge (\exists y.bad(y))$.

A Quick Summary

These are the elements of Predicate Logic's syntax.

1. Variables: x, y, z — we will mostly use these 3 alphabets to represent variables; if we need more, we will mention explicitly.
2. Objects: *ada, cat, rain, ...* any lower case term that is not x, y or z and is not followed by open and close parentheses.
3. Predicates: *works*($_$, $_$), *knows*($_$, $_$), *good*($_$).
4. Connectives: $\neg, \wedge, \vee, \implies, \iff$.
5. Quantifiers: $\forall$ and $\exists$.

Free Variables

A sentence is *grounded* iff there is no free variable in it. The set of free variables appearing in a sentence is defined as follows

$$\begin{aligned}fv(x) &= \{x\} \\fv(object) &= \{\} \\fv(p(\psi_1, \dots, \psi_n)) &= fv(\psi_1) \cup \dots \cup fv(\psi_n) \\fv(\psi \wedge \phi) &= fv(\psi) \cup fv(\phi) \\fv(\psi \vee \phi) &= fv(\psi) \cup fv(\phi) \\fv(\psi \implies \phi) &= fv(\psi) \cup fv(\phi) \\fv(\psi \iff \phi) &= fv(\psi) \cup fv(\phi) \\fv(\neg\psi) &= fv(\psi) \\fv(\forall x.\psi) &= fv(\psi) - \{x\} \\fv(\exists x.\psi) &= fv(\psi) - \{x\}\end{aligned}$$

$fv(\psi)$ denotes a meta function that extracts the free variables from a sentence ψ .

Free Variables: An Example

For instance, consider

$$\begin{aligned}fv(\exists z.(knows(ada, z) \wedge knows(cara, z))) &= \\fv((knows(ada, z) \wedge knows(cara, z))) - \{z\} &= \\(fv(knows(ada, z)) \cup fv(knows(cara, z))) - \{z\} &= \\(\{z\} \cup \{z\}) - \{z\} &= \\\{\} &= \end{aligned}$$

Therefore $\exists z.(knows(ada, z) \wedge knows(cara, z))$ is a ground sentence.

On the other hand, $good(x)$ is not a ground sentence.

Semantics: Unquantified Ground Sentences

The semantics for this subset is the same as the one for propositional logic, i.e. we can evaluate a non-quantified ground sentence based on a particular truth-assignment.

For instance if $knows(ada, cara)$ is 1 and $knows(cara, daphne)$ is 0, then evaluating $knows(ada, cara) \vee knows(cara, daphne)$ returns 1.

Instantiation

To understand the semantics of quantified ground sentences, we need an additional definition.

An **instance** of a sentence is a version in which all free variables are replaced by ground terms. In the current version the only possible ground terms are objects.

For example an instance of $knows(ada, z)$ is $knows(ada, cara)$.

Evaluation of Quantified Sentences

- ▶ A **universal** quantified sentence is evaluated to 1 for a truth assignment if and only if **every** instance of the scope of the quantified sentence is 1 for that assignment.
- ▶ An **existential** quantified sentence is evaluated to 1 for a truth assignment if and only if **some** instance of the scope of the quantified sentence is 1 for that assignment.

Evaluation: An Example

For example, let $Obj = \{ada, beatrice, cara, daphne\}$ and suppose the truth assignment contains

$$knows(ada, cara) = 1, \quad knows(cara, cara) = 0,$$

with the remaining values determined by the assignment. Then

$$\exists z. knows(ada, z)$$

is evaluated to 1, because the instance $knows(ada, cara)$ is 1.

Evaluation: Another Example

In contrast,

$$\forall z. \textit{knows}(\textit{cara}, z)$$

is evaluated to 0 because at least one instance, namely $\textit{knows}(\textit{cara}, \textit{cara})$, is 0.

Evaluation therefore asks whether a quantified sentence is true under one particular truth assignment.

Evaluation: “Everyone Knows Someone”

Another example, consider the truth assignment

$$\begin{aligned} \textit{knows}(\textit{ada}, \textit{cara}) &= 1, & \textit{knows}(\textit{cara}, \textit{beatrice}) &= 1, \\ \textit{knows}(\textit{beatrice}, \textit{daphne}) &= 1, & \textit{knows}(\textit{daphne}, \textit{ada}) &= 1 \end{aligned}$$

under which the following sentence stating “everyone knows someone”

$$\forall x. \exists y. \textit{knows}(x, y)$$

evaluates to 1.

Satisfaction

- ▶ A truth assignment satisfies a sentence with free variables if and only if it satisfies **every** instance of that sentence.
- ▶ A truth assignment satisfies a set of sentences if and only if it satisfies **every** sentence in the set.

Satisfaction: An Example

For example, consider the sentence with one free variable

$$\textit{knows}(x, \textit{cara}).$$

A truth assignment satisfies this sentence only if every instance is true. In particular, it must satisfy all of

$$\textit{knows}(\textit{ada}, \textit{cara}), \quad \textit{knows}(\textit{beatrice}, \textit{cara}), \quad \textit{knows}(\textit{cara}, \textit{cara}), \quad \textit{knows}(\textit{daphne}, \textit{cara}).$$

If even one of these instances is 0, the assignment does not satisfy $\textit{knows}(x, \textit{cara})$.

Satisfaction: A Set of Sentences

Similarly, an assignment satisfies the set

$$\{ \textit{knows}(x, \textit{cara}), \exists z. \textit{knows}(\textit{ada}, z) \}$$

only when it satisfies both sentences: every instance of the first sentence must be 1, and at least one instance of the second sentence must be 1.

Satisfaction therefore describes whether a truth assignment meets a sentence or a collection of sentences.

Defining New Predicates

One very cool idea of predicate logic is to allow new predicates to be defined in terms of the existing ones.

For example, if we want to define a predicate “connector”. A person is a connector if she knows everyone. Instead of enumerating how a connector person knows another person via the *knows*(_,_) predicate, we could write

$$\forall x.(\text{connector}(x) \iff (\forall y.\text{knows}(x,y)))$$

Proof System: Natural Deduction

The Natural Deduction proof system for predicate logic is a natural extension of the propositional logic one.

Besides $(\implies \text{Elim})$, $(\implies \text{Intro})$, $(\wedge \text{Elim})$, $(\wedge \text{Intro})$, $(\vee \text{Elim})$, $(\vee \text{Intro})$, $(\neg \text{Elim})$, $(\neg \text{Intro})$, $(\iff \text{Elim})$, $(\iff \text{Intro})$, we have additional rules for the quantifiers.

The propositional rules keep the same meaning when their propositions are predicate-logic formulas. For example, each of the following is a valid use of the corresponding rule.

Implication Elimination ($\implies$ Elim)

From $\text{knows}(\text{ada}, \text{cara}) \implies \text{trusts}(\text{cara}, \text{ada})$ and $\text{knows}(\text{ada}, \text{cara})$, derive $\text{trusts}(\text{cara}, \text{ada})$.

Implication Introduction ($\Rightarrow$ Intro)

If a sub-proof temporarily assumes $knows(x, y)$ and derives $knows(y, x)$, the assumption can be discharged to derive $knows(x, y) \Rightarrow knows(y, x)$.

Conjunction Elimination ($\wedge$ Elim)

From $knows(ada, cara) \wedge knows(ada, daphne)$, derive $knows(ada, cara)$ or derive $knows(ada, daphne)$.

Conjunction Introduction ($\wedge$ Intro)

From $knows(ada, cara)$ and $knows(cara, daphne)$, derive $knows(ada, cara) \wedge knows(cara, daphne)$.

Disjunction Elimination ($\vee$ Elim)

From $knows(ada, cara) \vee knows(ada, daphne)$, together with

$$knows(ada, cara) \implies connected(ada)$$

and

$$knows(ada, daphne) \implies connected(ada),$$

derive $connected(ada)$. Both alternatives lead to the same conclusion.

Disjunction Introduction ($\vee$ Intro)

From $knows(ada, cara)$, derive $knows(ada, cara) \vee knows(ada, daphne)$. It is enough to establish one disjunct.

Negation Elimination ($\neg$ Elim)

From $\neg\neg\textit{knows}(\textit{ada}, \textit{cara})$, derive $\textit{knows}(\textit{ada}, \textit{cara})$.

Negation Introduction ($\neg$ Intro)

If assuming *follows*(*ada*, *ada*) leads both to *p* and to $\neg p$, derive $\neg \textit{follows}(\textit{ada}, \textit{ada})$. The contradictory consequences show that the assumption cannot hold.

Biconditional Elimination ($\iff$ Elim)

From

$$\text{connector}(x) \iff \forall y. \text{knows}(x, y),$$

derive either direction:

$$\text{connector}(x) \implies \forall y. \text{knows}(x, y)$$

or

$$(\forall y. \text{knows}(x, y)) \implies \text{connector}(x).$$

Biconditional Introduction ($\iff$ Intro)

If both directions have been proved,

$$\text{connector}(x) \implies \forall y. \text{knows}(x, y)$$

and

$$(\forall y. \text{knows}(x, y)) \implies \text{connector}(x),$$

then derive

$$\text{connector}(x) \iff \forall y. \text{knows}(x, y).$$

Universal Elimination ($\forall$ Elim)

$$\frac{\forall x.\psi \quad \tau \text{ is ground}}{[\tau/x]\psi}$$

Where $[\tau/x]\psi$ reads as “substituting all occurrences of the free variable x in ψ by τ ”.

Substitution

The substitution operation is defined as follows.

$$\begin{array}{lll} [\tau/x]x & = & \tau \\ [\tau/x]\psi \wedge \phi & = & ([\tau/x]\psi) \wedge ([\tau/x]\phi) \\ [\tau/x]\psi \vee \phi & = & ([\tau/x]\psi) \vee ([\tau/x]\phi) \\ [\tau/x]\neg\psi & = & \neg([\tau/x]\psi) \\ [\tau/x]\psi \implies \phi & = & ([\tau/x]\psi) \implies ([\tau/x]\phi) \\ [\tau/x]\psi \iff \phi & = & ([\tau/x]\psi) \iff ([\tau/x]\phi) \\ [\tau/x]\forall y.\psi & = & \forall y.([\tau/x]\psi) & \text{if } x \neq y \wedge y \notin \text{fv}(\tau) \\ [\tau/x]\exists y.\psi & = & \exists y.([\tau/x]\psi) & \text{if } x \neq y \wedge y \notin \text{fv}(\tau) \end{array}$$

Substitution: Side Conditions

In the last two cases, we have a side condition if $x \neq y \wedge y \notin \text{fv}(\tau)$; when the side condition is not met, we have to rename y consistently in ψ so that it won't clash with x and free variables in τ .

The above substitution operation is (overly) generalized to handle scenarios where τ might contain free variables. In the current context, τ is always grounded.

$\forall$ Elim: Intuition

The Universal Elimination rule says that if $\forall x.\psi$ is true, ψ should remain true regardless of what ground instance we give to x .

For example, from

$$\forall x. \textit{knows}(x, \textit{cara})$$

we may instantiate x with any ground object in Obj . In particular, we may derive

$$\textit{knows}(\textit{ada}, \textit{cara}), \quad \textit{knows}(\textit{beatrice}, \textit{cara}), \quad \textit{knows}(\textit{daphne}, \textit{cara}).$$

The chosen term must be ground. Thus, Universal Elimination does not permit us to replace x with another free variable when the rule requires a ground instance.

$\forall$ Elim: A Proof

Let's consider another example (adopted from Stanford's course), suppose we have

$$\forall y. \exists z. \textit{follows}(y, z)$$

and

$$\forall x. \forall y. (\exists z. \textit{follows}(y, z) \implies \textit{follows}(x, y))$$

as premises. We want to verify that $\textit{follows}(\textit{ada}, \textit{cara})$.

$\forall$ Elim: A Proof (Cont'd)

The first premise means “each person must follow someone”. The second premise means “for a person to follow a followee, that followee must follow someone also”.

We establish the proof as follows

1.	$\forall y. \exists z. \text{follows}(y, z)$	Premise
2.	$\forall x. \forall y. (\exists z. \text{follows}(y, z) \implies \text{follows}(x, y))$	Premise
3.	$\exists z. \text{follows}(\text{cara}, z)$	$\forall$ Elim : 1
4.	$\forall y. (\exists z. \text{follows}(y, z) \implies \text{follows}(\text{ada}, y))$	$\forall$ Elim : 2
5.	$\exists z. \text{follows}(\text{cara}, z) \implies \text{follows}(\text{ada}, \text{cara})$	$\forall$ Elim : 4
6.	$\text{follows}(\text{ada}, \text{cara})$	$\implies$ Elim : 5, 3

Universal Introduction ($\forall$ Intro)

$$\frac{\psi \quad \tau \text{ is not used in any active assumption}}{\forall x.[x/\tau]\psi}$$

The premise τ is not used in any active assumption means that the sub predicate τ inside ψ is not used as premises nor as a substitution in any preceding step on the current level and parent level of the proof.

$\forall$ Intro: An Example

For example, let a be a fresh object symbol that does not occur in any active assumption. If a sub-proof derives

$$\textit{knows}(a, \textit{cara}),$$

without using any fact that is special to a , then we may generalize the fresh symbol and conclude:

$$\forall x. \textit{knows}(x, \textit{cara}).$$

$\forall$ Intro: Freshness Matters

The freshness condition is essential. From the assumption $knows(ada, cara)$, we may derive that same fact, but we may **not** conclude $\forall x. knows(x, cara)$, because the proof used a fact specifically about Ada rather than an arbitrary object.

$\forall$ Intro: A Stronger Conclusion

Let's consider another example. We start from the same premises as the last example (in the $\forall$ Elim rule), and want to verify a stronger conclusion

$$\forall x.\forall y.\textit{follows}(x, y)$$

$\forall$ Intro: A Stronger Conclusion (Cont'd)

1.	$\forall y. \exists z. \text{follows}(y, z)$	Premise
2.	$\forall x. \forall y. (\exists z. \text{follows}(y, z) \implies \text{follows}(x, y))$	Premise
3.	$\exists z. \text{follows}(d, z)$	$\forall\text{Elim} : 1$
4.	$\forall y. (\exists z. \text{follows}(y, z) \implies \text{follows}(c, y))$	$\forall\text{Elim} : 2$
5.	$\exists z. \text{follows}(d, z) \implies \text{follows}(c, d)$	$\forall\text{Elim} : 4$
6.	$\text{follows}(c, d)$	$\implies \text{Elim} : 5, 3$
7.	$\forall y. \text{follows}(c, y)$	$\forall\text{Intro} : 6$
8.	$\forall x. \forall y. \text{follows}(x, y)$	$\forall\text{Intro} : 7$

Existential Introduction ($\exists$ Intro)

$$\frac{\psi \quad \tau \text{ is ground}}{\exists x. [x/\tau]\psi}$$

This rule says that if a predicate holds for one particular ground term, then there exists an object for which the predicate holds. The term used in the premise becomes the **witness** for the existential statement.

∃Intro: An Example

For example, from

$$\textit{knows}(\textit{ada}, \textit{cara})$$

we may use *ada* as a witness and conclude:

$$\exists x. \textit{knows}(x, \textit{cara}).$$

The rule does not require us to identify every object satisfying the predicate; one valid witness is sufficient.

Existential Elimination ($\exists$ Elim)

$$\frac{\begin{array}{c} \exists x. \psi \\ [\tau/x]\psi \vdash \phi \\ \tau \text{ is not used in any active assumption or in } \phi \end{array}}{\phi}$$

This rule allows us to reason about an object whose existence is guaranteed, without needing to know which object it is. We introduce a fresh ground term τ as a temporary name for an arbitrary witness, derive the desired conclusion, and then discharge that temporary name.

$\exists$ Elim: An Example

For example, suppose we have the following premises

$$\exists x. \textit{knows}(x, \textit{cara})$$

and

$$\forall x. (\textit{knows}(x, \textit{cara}) \implies \textit{connected}(\textit{cara})).$$

We want to show that $\textit{connected}(\textit{cara})$.

$\exists$ Elim: A Sub-proof

Using a fresh witness a , the sub-proof is:

1.	$\exists x. \text{knows}(x, \text{cara})$	Premise
2.	$\forall x. (\text{knows}(x, \text{cara}) \implies \text{connected}(\text{cara}))$	Premise
3.1.	$\text{knows}(a, \text{cara})$	a is not used in any active assumption or in $\text{connected}(\text{cara})$
3.2.	$\text{knows}(a, \text{cara}) \implies \text{connected}(\text{cara})$	$\forall$ Elim 2
3.3.	$\text{connected}(\text{cara})$	$\implies$ Elim 3.1, 3.2
3.	$\text{connected}(\text{cara})$	$\exists$ Elim : 3.1 – 3.3

Further Readings

- ▶ http://intrologic.stanford.edu/sections/section_07.html?section=6
- ▶ http://intrologic.stanford.edu/sections/section_07.html?section=7
- ▶ http://intrologic.stanford.edu/sections/section_07.html?section=8

First-Order Logic

Recall that we imposed the following syntax restrictions to predicate logic.

1. Objects must be finite
2. Predicate's argument must be a simple object or variable

In this section we consider an extension to our predicate logic where the above two restrictions are lifted. We call it **First-Order Logic**.

In the Stanford course, such an extension is called Term Logic or Herbrand Logic, which has some subtle differences from the actual first-order logic. In this course, we treat first-order logic and Herbrand logic as the same for the ease of understanding.

Terms

After lifting the two syntax restrictions, we are allowed to write sentences such as

loves(father(cara), mother(cara))

In the above, we have nested predicates. In the presence of this complexity, we divide the predicates into three types of **terms**.

1. **Atom objects** (AKA simple terms): *cara*, *ada*, ...
2. **Functions**, which take a term and return a term, e.g. *father*(_) — the father of Cara is a person, not a true or false value.
3. **Relations**, which take a term and return a truth-value, e.g. *loves*(_, _)

The sentence above means “Cara’s father loves Cara’s mother”.

Peano Arithmetic

Suppose we want to model natural number arithmetic. In our system we have the following language

1. **Atom:** *zero* (which stands for the number 0, instead of the logical false value.)
2. **Function:** *suc*(_) returns the next natural number, for instance, *suc*(*zero*) is 1 (the number 1, not the logical true value.), *suc*(*suc*(*zero*)) is 2, ...
3. **Relations:** *even*(_) and *odd*(_). *even*(*zero*), *odd*(*suc*(*zero*)), ...

Now the cool thing is that, without the need to enumerate all the *even* and *odd* relations with ground terms, we can define them recursively as follows.

Peano Arithmetic: Axioms

- ▶ Base cases (EvenZero) and (NegOddZero)

$$\begin{aligned} & \text{even}(\text{zero}) \\ & \neg \text{odd}(\text{zero}) \end{aligned}$$

- ▶ Recursive cases (EvenSuc) and (OddSuc)

$$\begin{aligned} \forall x. (\text{odd}(\text{suc}(x)) &\iff \text{even}(x)) \\ \forall x. (\text{even}(\text{suc}(x)) &\iff \text{odd}(x)) \end{aligned}$$

The above definitions are called **axioms**. We can think of axioms as the “global” premises, which are assumed to be valid.

Semantics of First-Order Logic

The semantics of first-order logic is a faithful extension of the one we explained earlier.

- ▶ Truth values are assigned to all the ground predicates (i.e. relation being applied to ground terms).
- ▶ However evaluation and satisfaction using truth-tables becomes immediately infeasible because of the infinite number of ground terms.
- ▶ The only practical approach is via **proof**.

Proof System of First-Order Logic

The good news is that there is almost no change to the proof system. All the rules that we defined earlier are still applicable.

Peano Arithmetic: A Simple Proof

Our goal is to make use of the axioms (EvenZero), (NegOddZero), (EvenSuc) and (OddSuc), to verify the following

$$\text{even}(\text{suc}(\text{suc}(\text{zero})))$$

i.e. “2 is an even number”.

To make our proof easier, let's derive some sub-lemmas from the axioms.

Lemma 1: “the Successor of an Even Number is Odd”

$$\forall x. (even(x) \implies odd(suc(x)))$$

The proof goes as follows

1.	$\forall x. (odd(suc(x)) \iff even(x))$	Axiom(OddSuc)
2.	$odd(suc(n)) \iff even(n)$	$\forall$ Elim : 1
3.	$even(n) \implies odd(suc(n))$	$\iff$ Elim : 2
4.	$\forall x. (even(x) \implies odd(suc(x)))$	$\forall$ Intro : 3

Lemma 2: “the Successor of an Odd Number is Even”

$$\forall x. (odd(x) \implies even(suc(x)))$$

The proof goes as follows

1.	$\forall x. (even(suc(x)) \iff odd(x))$	Axiom(EvenSuc)
2.	$even(suc(m)) \iff odd(m)$	$\forall$ Elim : 1
3.	$odd(m) \implies even(suc(m))$	$\iff$ Elim : 2
4.	$\forall x. odd(x) \implies even(suc(x))$	$\forall$ Intro : 3

Main Proof: “2 is an Even Number”

1.	$even(zero)$	Axiom(EvenZero)
2.	$\forall x.(even(x) \implies odd(suc(x)))$	Lemma1
3.	$\forall x.(odd(x) \implies even(suc(x)))$	Lemma2
4.	$even(zero) \implies odd(suc(zero))$	$\forall$ Elim : 2
5.	$odd(suc(zero))$	$\implies$ Elim : 1, 4
6.	$odd(suc(zero)) \implies even(suc(suc(zero)))$	$\forall$ Elim : 3
7.	$even(suc(suc(zero)))$	$\implies$ Elim : 5, 6

“3 is an Odd Number”

If we continue from the above proof, we can also prove “3 is an odd number”

8.	$even(suc(suc(zero))) \implies odd(suc(suc(suc(zero))))$	$\forall\text{Elim} : 2$
9.	$odd(suc(suc(suc(zero))))$	$\implies \text{Elim} : 7, 8$

Limitation of Proof by Deduction

However the deduction proof can only prove **ground** predicates. We can't use it to prove a general term.

Suppose we want to verify that “adding 2 to an even number gives us an even number; adding 2 to an odd number gives us an odd number.” Formally, our goal is to show that

$$\forall x.((\text{even}(x) \implies \text{even}(\text{suc}(\text{suc}(x)))) \wedge (\text{odd}(x) \implies \text{odd}(\text{suc}(\text{suc}(x)))))$$

It is impossible to enumerate all the objects in the natural numbers. We need to consider a sound-and-complete version of the induction.

Linear Induction

Linear Induction works when the domain of terms consists of an atom object o and a unary function f that takes a term and returns the next term.

$$\frac{\psi(o) \quad \forall x.(\psi(x) \implies \psi(f(x)))}{\forall x.\psi(x)}$$

- ▶ The first premise in the above inference rule is called the *base case*.
- ▶ The 2nd premise is called the *inductive case*.
- ▶ The antecedent of the inductive case is called the *inductive hypothesis*, and the consequent is called the *inductive conclusion*.

Applying Linear Induction

Let's apply the above inference rule to the sentence being considered.

$$\frac{\begin{array}{c} \text{even}(\text{zero}) \implies \text{even}(\text{suc}(\text{suc}(\text{zero}))) \wedge \text{odd}(\text{zero}) \implies \text{odd}(\text{suc}(\text{suc}(\text{zero}))) \\ \text{even}(\text{suc}(\text{zero})) \implies \text{even}(\text{suc}(\text{suc}(\text{suc}(\text{zero})))) \wedge \text{odd}(\text{suc}(\text{zero})) \implies \text{odd}(\text{suc}(\text{suc}(\text{suc}(\text{zero})))) \\ ((\text{even}(x) \implies \text{even}(\text{suc}(\text{suc}(x)))) \wedge (\text{odd}(x) \implies \text{odd}(\text{suc}(\text{suc}(x))))) \\ \implies \\ ((\text{even}(\text{suc}(x)) \implies \text{even}(\text{suc}(\text{suc}(\text{suc}(x))))) \wedge (\text{odd}(\text{suc}(x)) \implies \text{odd}(\text{suc}(\text{suc}(\text{suc}(x))))) \end{array}}{\forall x. ((\text{even}(x) \implies \text{even}(\text{suc}(\text{suc}(x)))) \wedge (\text{odd}(x) \implies \text{odd}(\text{suc}(\text{suc}(x)))))}$$

Base Case: zero

The proof of the predicate

$$\text{even}(\text{zero}) \implies \text{even}(\text{suc}(\text{suc}(\text{zero})))$$

is identical to the above “simple proof”, except that $\text{even}(\text{zero})$ is a premise instead of an axiom.

The proof of the predicate

$$\text{odd}(\text{zero}) \implies \text{odd}(\text{suc}(\text{suc}(\text{zero})))$$

is obviously valid because $\text{odd}(\text{zero})$ is false.

Base Case: $\text{suc}(\text{zero})$

To finish this base case we first record that $1 = \text{suc}(\text{zero})$ is not even.

1.	$\neg \text{odd}(\text{zero})$	Axiom(NegOddZero)
2.	$\forall x. \text{even}(\text{suc}(x)) \iff \text{odd}(x)$	Axiom(EvenSuc)
3.	$\text{even}(\text{suc}(\text{zero})) \iff \text{odd}(\text{zero})$	$\forall\text{Elim} : 2$
4.	$\text{even}(\text{suc}(\text{zero})) \implies \text{odd}(\text{zero})$	$\iff \text{Elim} : 3$
5.1.	$\text{even}(\text{suc}(\text{zero}))$	Assumption
5.2.	$\text{odd}(\text{zero})$	$\implies \text{Elim} : 4, 5.1$
5.3.	$\neg \text{odd}(\text{zero})$	Axiom(NegOddZero)
5.	$\neg \text{even}(\text{suc}(\text{zero}))$	$\neg\text{Intro} : 5.1 - 5.3$

Base Case: $\text{suc}(\text{zero})$ (Cont'd)

From $\neg \text{even}(\text{suc}(\text{zero}))$ the antecedent of $\text{even}(\text{suc}(\text{zero})) \implies \text{even}(\text{suc}(\text{suc}(\text{suc}(\text{zero}))))$ is false, so that implication holds vacuously and the first conjunct of the base case at $\text{suc}(\text{zero})$ is discharged.

The predicate $\text{odd}(\text{suc}(\text{zero})) \implies \text{odd}(\text{suc}(\text{suc}(\text{suc}(\text{zero}))))$ is valid because we have verified “1 is an odd number” and “3 is an odd number” in the earlier section.

Inductive Case

Let x be arbitrary. The induction hypothesis is not needed explicitly here: the two parity lemmas, instantiated at successive terms, directly establish the required successor implication.

Inductive Case: The Proof

1.	$\forall y.(even(y) \implies odd(s(y)))$	Lemma 1
2.	$\forall y.(odd(y) \implies even(s(y)))$	Lemma 2
3.	$even(s(x)) \implies odd(s(s(x)))$	$\forall$ Elim : 1
4.	$odd(s(s(x))) \implies even(s(s(s(x))))$	$\forall$ Elim : 2
5.1.	$even(s(x))$	Assumption
5.2.	$odd(s(s(x)))$	$\implies$ Elim : 3, 5.1
5.3.	$even(s(s(s(x))))$	$\implies$ Elim : 4, 5.2
5.	$even(s(x)) \implies even(s(s(s(x))))$	$\implies$ Intro : 5.1 – 5.3
6.	$odd(s(x)) \implies even(s(s(x)))$	$\forall$ Elim : 2
7.	$even(s(s(x))) \implies odd(s(s(s(x))))$	$\forall$ Elim : 1
8.1.	$odd(s(x))$	Assumption
8.2.	$even(s(s(x)))$	$\implies$ Elim : 6, 8.1
8.3.	$odd(s(s(s(x))))$	$\implies$ Elim : 7, 8.2
8.	$odd(s(x)) \implies odd(s(s(s(x))))$	$\implies$ Intro : 8.1 – 8.3
9.	$(even(s(x)) \implies even(s(s(s(x)))) \wedge$ $(odd(s(x)) \implies odd(s(s(s(x))))$	$\wedge$ Intro : 5, 8

We use $s(_)$ as the shorthand for $suc(_)$.

Inductive Case: The Proof (Cont'd)

10.1.	$(\text{even}(x) \implies \text{even}(s(s(x)))) \wedge$ $(\text{odd}(x) \implies \text{odd}(s(s(x))))$	Assumption
10.2.	$(\text{even}(s(x)) \implies \text{even}(s(s(s(x)))) \wedge$ $(\text{odd}(s(x)) \implies \text{odd}(s(s(s(x))))$	Reiteration : 9
10.	$((\text{even}(x) \implies \text{even}(s(s(x)))) \wedge (\text{odd}(x) \implies \text{odd}(s(s(x)))))$ $\implies$ $((\text{even}(s(x)) \implies \text{even}(s(s(s(x)))) \wedge (\text{odd}(s(x)) \implies \text{odd}(s(s(s(x)))))$	$\implies$ Intro : 10.1 – 10.2
11.	$\forall x. \left(\begin{array}{c} ((\text{even}(x) \implies \text{even}(s(s(x)))) \wedge (\text{odd}(x) \implies \text{odd}(s(s(x))))) \\ \implies \\ ((\text{even}(s(x)) \implies \text{even}(s(s(s(x)))) \wedge (\text{odd}(s(x)) \implies \text{odd}(s(s(s(x))))) \end{array} \right)$	$\forall$ Intro : 10

Thus the inductive case is established. Together with the two base cases, linear induction proves the desired statement for every Peano number.

Tree Induction

Linear induction works because the Peano numerals form a **chain**: one base object *zero* and one unary function *suc* that steps from a number to its successor, so the inductive premise $\psi(x) \implies \psi(\text{suc}(x))$ moves along a single spine.

The family terms of the earlier example, *loves*(*father*(*cara*), *mother*(*cara*)), do not form a chain. From any person *p* there are *two* parents, *father*(*p*) and *mother*(*p*); from each of those there are two more; and so on. The ancestry of a person is therefore a **tree**:

p, *father*(*p*), *mother*(*p*), *father*(*father*(*p*)), *mother*(*father*(*p*)),
father(*mother*(*p*)), *mother*(*mother*(*p*)), ...

Tree Induction (Cont'd)

What matters is that this tree is *regular*: there is a single type (persons), and every node has exactly two successors (the father and the mother).

That regularity is precisely what separates this case from its two neighbours:

- ▶ It has more than one successor, so it lies beyond **linear** induction.
- ▶ Its branching is uniform, so it is more restricted than **structural** induction, where different constructors have different arities.

Induction over such a regular tree is **tree induction**, the intermediate form.

Tree Induction: The Principle

To keep the induction well founded, let the domain be the family of a base person r : r and everyone obtained from r by repeatedly applying *father* and *mother*. Every such person is finitely many steps from r , so the induction is well founded.

The principle is: to prove a property P of every person in this family, it suffices to prove the base case and the two branches, i.e. that the property is inherited from a person to their father and from a person to their mother:

$$\frac{\begin{array}{l} P(r) \\ \forall p. P(p) \implies P(\text{father}(p)) \\ \forall p. P(p) \implies P(\text{mother}(p)) \end{array}}{\forall p. P(p)}$$

Tree Induction: Positioning

The first line is the *base case*; the second and third are the *paternal* and *maternal branches*.

This is the intermediate position:

- ▶ **Linear** induction has one branch (a unary successor).
- ▶ **Tree** induction has a fixed number of same-typed branches (here two, the parents).
- ▶ **Structural** induction has one case per constructor with a variable number of recursive sub-terms.

Tree Induction: An Example

Now take $P(p)$ to be $\text{loves}(\text{father}(p), \text{mother}(p))$ — “the parents of p love each other” — and suppose the following axioms, where the second and third record the distinction between the paternal and the maternal side:

- ▶ **Base.** $\text{loves}(\text{father}(r), \text{mother}(r))$. The base person’s own parents love each other.
- ▶ **Paternal.**
 $\forall p. \text{loves}(\text{father}(p), \text{mother}(p)) \implies \text{loves}(\text{father}(\text{father}(p)), \text{mother}(\text{father}(p)))$.
If a person’s parents love each other, so do the paternal grandfather and the paternal grandmother.
- ▶ **Maternal.** $\forall p. \text{loves}(\text{father}(p), \text{mother}(p)) \implies \text{loves}(\text{father}(\text{mother}(p)), \text{mother}(\text{mother}(p)))$. If a person’s parents love each other, so do the maternal grandfather and the maternal grandmother.

Tree Induction: The Goal

Our goal is the general claim, for every p in the family,

$$P(p) \equiv \text{loves}(\text{father}(p), \text{mother}(p)),$$

“every father loves every mother, provided they are the parents of a common child.” We verify it by tree induction.

- ▶ **Base case.** $P(r) = \text{loves}(\text{father}(r), \text{mother}(r))$ holds by the Base axiom.
- ▶ **Inductive case.** Fix an arbitrary p and assume $P(p)$; both branches then follow from the axioms.

Tree Induction: The Proof

1.	$loves(father(p), mother(p))$	Assumption $P(p)$
2.	$\forall q. loves(father(q), mother(q)) \implies loves(father(father(q)), mother(father(q)))$	Axiom (Paternal)
3.	$loves(father(p), mother(p)) \implies loves(father(father(p)), mother(father(p)))$	$\forall$ Elim : 2
4.	$loves(father(father(p)), mother(father(p)))$	$\implies$ Elim : 3, 1
5.	$\forall q. loves(father(q), mother(q)) \implies loves(father(mother(q)), mother(mother(q)))$	Axiom (Maternal)
6.	$loves(father(p), mother(p)) \implies loves(father(mother(p)), mother(mother(p)))$	$\forall$ Elim : 5
7.	$loves(father(mother(p)), mother(mother(p)))$	$\implies$ Elim : 6, 1

Tree Induction: Conclusion

Step 4 is $P(\text{father}(p))$, discharging the paternal branch; step 7 is $P(\text{mother}(p))$, discharging the maternal branch. Both are derived from the single hypothesis $P(p)$.

Since the base case and both branches hold, tree induction gives $P(p)$ for every person p in the family: every father loves every mother, provided they share the same child.

This is the intermediate form — more branching than linear induction, but more regular than the structural case that follows.

Structural Induction

Tree induction handles *regular* trees, where every node has the same number of successors of the same type. Structural induction handles a grammar with several constructors and varying arities.

The terms of our arithmetic system are generated by

$$Term ::= zero \mid suc(Term) \mid add(Term, Term)$$

- ▶ *zero* has no recursive sub-terms.
- ▶ *suc* has one recursive sub-term.
- ▶ *add* has two recursive sub-terms.

Structural Induction: A Motivating Term

Consider

$$\mathit{add}(\mathit{suc}(\mathit{zero}), \mathit{add}(\mathit{suc}(\mathit{suc}(\mathit{zero})), \mathit{zero})).$$

The *add* node contains two independently constructed sub-terms. A proof about this term must use the property of both sub-terms in the *add* case.

Structural Induction: The Rule

Let $P(t)$ be a property of terms. To prove P for every term, establish one case for each constructor:

$$\frac{\begin{array}{c} P(\text{zero}) \\ \forall x. (P(x) \implies P(\text{suc}(x))) \\ \forall x. \forall y. ((P(x) \wedge P(y)) \implies P(\text{add}(x, y))) \end{array}}{\forall t. P(t)}$$

The *add* case supplies two induction hypotheses, one for each recursive argument.

A Normalisation Property

We use $eq(t, u)$ to mean that two terms denote the same natural number. AddSuc moves a successor from the left argument to the right:

$$add(suc(x), y) \longrightarrow add(x, suc(y)).$$

AddZero removes the zero left argument:

$$add(zero, y) = y.$$

Numerals

A numeral is a term of the form

$zero, \quad suc(zero), \quad suc(suc(zero)), \quad \dots$

Define $P(t)$ to mean:

There is a numeral n such that $eq(t, n)$.

We prove $\forall t. P(t)$ by structural induction.

Normalisation: Base Case

The base term *zero* is already a numeral. Eq witnesses the required equality:

$$eq(\text{zero}, \text{zero}).$$

Therefore $P(\text{zero})$ holds.

Normalisation: Successor Case

Assume $P(x)$. Then there is a numeral n such that $eq(x, n)$.

By equality replacement,

$$eq(suc(x), suc(n)).$$

Since $suc(n)$ is a numeral, $P(suc(x))$ follows.

Normalisation: Addition Case

Assume $P(x) \wedge P(y)$. Then there are numerals m and n such that

$$eq(x, m) \quad \text{and} \quad eq(y, n).$$

By equality replacement, it is enough to normalize $add(m, n)$.

Addition of Numerals

Auxiliary lemma:

For every pair of numerals m and n , there is a numeral k such that $eq(add(m, n), k)$.

Prove this lemma by linear induction on m .

- ▶ **Base case.** AddZero gives $eq(add(zero, n), n)$.
- ▶ **Inductive case.** AddSuc moves one successor from the left to the right, and the induction hypothesis normalizes the resulting addition.

Therefore $add(m, n)$, and hence $add(x, y)$, is equal to a numeral. This proves $P(add(x, y))$.

Structural Induction: Summary

The three constructor cases are:

1. $P(\text{zero})$
2. $P(x) \implies P(\text{suc}(x))$
3. $(P(x) \wedge P(y)) \implies P(\text{add}(x, y))$

The proof has one case for each constructor. A linear induction rule would provide only one hypothesis and would be insufficient for the binary *add* constructor.

A Worked Term

Let

$$t = \text{add}(\text{suc}(\text{zero}), \text{add}(\text{suc}(\text{suc}(\text{zero})), \text{zero})).$$

The right sub-term normalizes as follows:

$$\begin{aligned} & \text{add}(\text{suc}(\text{suc}(\text{zero})), \text{zero}) \\ \longrightarrow & \text{add}(\text{suc}(\text{zero}), \text{suc}(\text{zero})) \\ \longrightarrow & \text{add}(\text{zero}, \text{suc}(\text{suc}(\text{zero}))) \\ = & \text{suc}(\text{suc}(\text{zero})). \end{aligned}$$

A Worked Term (Cont'd)

The whole term then becomes

$$add(suc(zero), suc(suc(zero))) \longrightarrow add(zero, suc(suc(suc(zero)))) = suc(suc(suc(zero))).$$

Thus $P(t)$ holds.

Structural Induction Compared

- ▶ **Linear induction** walks a chain: one base constructor and one unary successor.
- ▶ **Tree induction** walks a regular tree: a fixed number of same-typed branches.
- ▶ **Structural induction** handles a grammar with several constructors and varying arities.

Multidimension Induction

We now consider a property of two natural numbers. Prove

$$\forall x. \forall y. (eq(x, y) \implies ((even(x) \wedge even(y)) \vee (odd(x) \wedge odd(y)))).$$

This says that equal Peano numbers have the same parity.

Multidimension Induction: Pair Property

Let

$$P(x, y) \equiv eq(x, y) \implies ((even(x) \wedge even(y)) \vee (odd(x) \wedge odd(y))).$$

We use induction over the pair (x, y) .

Pair-Induction: Base Case

For $(zero, zero)$, Eq and $even(zero)$ give the result immediately:

$$eq(zero, zero) \wedge even(zero) \wedge even(zero).$$

Pair-Induction: Mixed Cases

If $x = \text{zero}$ and $y = \text{suc}(y')$, NEqBase gives

$$\neg \text{eq}(\text{zero}, \text{suc}(y')).$$

Similarly, NEqBase gives $\neg \text{eq}(\text{suc}(x'), \text{zero})$.

Both mixed cases satisfy P vacuously.

Pair-Induction: Successor Case

Assume $P(x', y')$ and suppose

$$eq(suc(x'), suc(y')).$$

By contrapositive of NEqRec, this implies $eq(x', y')$. The induction hypothesis gives

$$(even(x') \wedge even(y')) \vee (odd(x') \wedge odd(y')).$$

Pair-Induction: Parity Preservation

The parity axioms switch the parity of both successors:

$$\text{even}(\text{suc}(x')) \iff \text{odd}(x') \quad \text{and} \quad \text{odd}(\text{suc}(x')) \iff \text{even}(x').$$

The corresponding equations hold for y' . Thus equal predecessors with the same parity yield equal successors with the same parity.

Multidimension Induction: Conclusion

The base, mixed, and successor cases establish

$$\forall x. \forall y. (eq(x, y) \implies ((even(x) \wedge even(y)) \vee (odd(x) \wedge odd(y)))).$$

The proof reasons about two recursive dimensions simultaneously: the pair (x, y) .

Further Reading

1. http://intrologic.stanford.edu/chapters/chapter_11.html
2. http://intrologic.stanford.edu/chapters/chapter_12.html
3. http://intrologic.stanford.edu/chapters/chapter_13.html